U4 - REPASO o DML (Lenguaje de Manipulación de Datos)

- **Autor**: José Ángel Alejo Sillero
- **Centro**: Instituto Tecnológico Pablo de la Torre

- **Curso**: 2025/2026
- **Módulo**: Bases de Datos
- **Unidad**: 4 - DML (Lenguaje de Manipulación de Datos)

- U4 - REPASO o DML (Lenguaje de Manipulación de Datos)
  - **1. CONCEPTOS FUNDAMENTOS - DML**
    - Concepto base
  - **2. SENTENCIA SELECT - CONCEPTOS BÁSICOS**
    - Qué es SELECT
    - Funciones principales
  - **3. SINTAXIS BÁSICA DE SELECT**
    - Componentes
    - Ejemplos prácticos
  - **4. ALIAS**
    - ¿Qué es un alias?
    - Sintaxis de alias
    - **Aplicación de alias**
  - **5. OPERADOR DE CONCATENACIÓN - CONCAT()**
    - Función de concatenación
    - Sintaxis de CONCAT
    - Aplicación de CONCAT
  - **6. FILAS DUPLICADAS - DISTINCT**
    - Eliminación de duplicados
    - Sintaxis de DISTINCT
    - Aplicación de DISTINCT
  - **7. OPERADORES ARITMÉTICOS**
    - Tipos disponibles
    - **Reglas de precedencia**
    - Uso en consulta
  - **8. RESTRICCIÓN DE FILAS - WHERE**
    - Introducción a WHERE
    - Construcción de WHERE
  - **9. OPERADORES DE COMPARACIÓN**
    - Comparaciones básicas
    - Comparaciones avanzadas
  - **10. OPERADORES LÓGICOS**
  - 11. **ORDEN DE EVALUACIÓN DE OPERADORES**
  - **12. ORDENACIÓN - ORDER BY**
    - Definición de ORDER BY
    - Sintaxis para ordenación
    - Opciones disponibles
  - **13. CONSULTAS RESUMEN - FUNCIONES DE AGREGACIÓN**
    - Definición de agregación
    - Función de agregacion según tipo
    - Variantes de COUNT()
    - Ejemplo de agregación
  - **14. AGRUPAMIENTO - GROUP BY**
    - Agrupamiento de datos
    - Construcción de GROUP BY
    - Reglas importantes
    - Caso 1 - Una columna
    - Caso 2 - Varias columnas
  - **15. FILTRADO DE GRUPOS - HAVING**
    - Filtrado de grupos
    - Construcción de HAVING
    - Diferencia: WHERE vs HAVING
    - Ejemplo de HAVING
  - **16. CONSULTAS SOBRE VARIAS TABLAS - JOINS**
    - Uniones de tablas
    - Clasificación de uniones disponibles
      - A. COMPOSICIONES INTERNAS (INNER JOIN)
        - 1. NATURAL JOIN (automático)
        - 2. USING (columnas comunes)
        - 3. ON (máxima flexibilidad)
        - 4. CROSS JOIN (producto cartesiano)
      - B. COMPOSICIONES EXTERNAS (OUTER JOIN)
        - 1. LEFT JOIN (tabla izquierda)
        - 2. RIGHT JOIN (tabla derecha)
  - **17. SUBCONSULTAS**
    - Definición de subconsultas
    - Clasificación de subconsultas
    - A. EN LA CLÁUSULA WHERE
    - B. EN LA CLÁUSULA SELECTºº

        - C. OPERADORES DISPONIBLES
            - 1. Comparación básica
            - 2. IN y NOT IN
            - 3. ALL y ANY
            - 4. EXISTS y NOT EXISTS
    - **18. COMBINACIÓN DE CONSULTAS - UNION**
        - Definición de UNION
        - Restricciones de combinación
        - Variante 1 - UNION (Sin duplicados)
        - Variante 2 - UNION ALL (Con duplicados)
    - **19. FLUJO DE EJECUCIÓN DE SELECT**
        - Caso de estudio completo
    - **20. RESUMEN DE CONSULTAS DML**
    - **21. QUIZ RÁPIDO - REPASO**

---

1. CONCEPTOS FUNDAMENTOS - DML

Operación	Función	Acción
SELECT	Consulta datos	Extrae información
INSERT	Inserta datos	Añade registros
UPDATE	Actualiza datos	Modifica registros
DELETE	Elimina datos	Borra registros

Concepto base

El Lenguaje de Manipulación de Datos (DML) permite manipular el contenido de las bases de datos:

- Inserción de datos
- Actualización de datos
- Eliminación de datos
- Consulta de datos (enfoque principal en este tema)

---

2. SENTENCIA SELECT - CONCEPTOS BÁSICOS

Qué es SELECT

Sentencia que recupera información de la base de datos desde una o más tablas o vistas.

Funciones principales

Función	Descripción	Acción
Proyección	Selecciona determinadas columnas	Elige qué columnas mostrar
Selección	Selecciona determinadas filas	Filtra qué registros mostrar
Unión	Reúne datos de varias tablas	Combina información relacionada

---

3. SINTAXIS BÁSICA DE SELECT

```
SELECT nombre_columna1, nombre_columna2, ...
FROM nombre_tabla;
```

Componentes

- SELECT: Indica qué columnas mostrar
- FROM: Especifica la tabla de origen
- * (comodín): Muestra TODAS las columnas de la tabla

Ejemplos prácticos

```
-- Seleccionar todas las columnas
SELECT * FROM ciudad;

-- Seleccionar columnas específicas
SELECT Nombre, Poblacion, CodigoPais FROM ciudad;
```

---

4. ALIAS

¿Qué es un alias?

Renombre temporal de una columna o tabla en una consulta para mejorar la legibilidad del resultado.

Sintaxis de alias

```
SELECT nombre_columna AS alias_descriptivo
FROM nombre_tabla;
```

Aplicación de alias

```
SELECT Name AS Ciudad, Population AS Población, CountryCode AS "Codigo País"
FROM city;
```

---

5. OPERADOR DE CONCATENACIÓN - CONCAT()

Función de concatenación

Combina dos o más columnas o valores en una única columna de salida.

Sintaxis de CONCAT

```
SELECT CONCAT(columna1, 'separador', columna2) AS alias
FROM nombre_tabla;
```

Aplicación de CONCAT

```
SELECT CONCAT(Name, '_', Continent, '_', Population) AS Concatenacion
FROM country;
```

---

6. FILAS DUPLICADAS - DISTINCT

Eliminación de duplicados

Elimina filas duplicadas del resultado de una consulta, mostrando solo valores únicos.

Sintaxis de DISTINCT

```
SELECT DISTINCT columna1, columna2, ...
FROM nombre_tabla;
```

Aplicación de DISTINCT

```
SELECT DISTINCT Continent, Name FROM country;
```

---

7. OPERADORES ARITMÉTICOS

Tipos disponibles

Operador	Símbolo	Ejemplo
Suma	`+`	`5 + 3 = 8`
Resta	`-`	`10 - 2 = 8`
Multiplicación	`*`	`4 * 2 = 8`
División	`/`	`16 / 2 = 8`
Módulo	`%`	`10 % 3 = 1`

Reglas de precedencia

1. Multiplicación y División (evaluadas primero)
2. Suma y Resta (evaluadas después)
3. Misma prioridad: Evaluación de izquierda a derecha
4. Paréntesis: Fuerzan prioridad

Uso en consulta

```
SELECT 10 / (2 * (5 + 32)) AS Resultado;
```

---

8. RESTRICCIÓN DE FILAS - WHERE

Introducción a WHERE

Filtra filas según una o más condiciones, devolviendo solo los registros que las cumplen.

Construcción de WHERE

```
SELECT columna1, columna2, ...
FROM nombre_tabla
WHERE expresión_lógica;
```

---

9. OPERADORES DE COMPARACIÓN

Comparaciones básicas

Operador	Significado	Ejemplo
`=`	Igual que	`edad = 30`
`<>` o `!=`	Distinto que	`ciudad <> 'Madrid'`
`>`	Mayor que	`precio > 100`
`<`	Menor que	`edad < 18`
`>=`	Mayor o igual	`edad >= 18`
`<=`	Menor o igual	`precio <= 50`

Comparaciones avanzadas

Operador	Significado	Ejemplo
`BETWEEN`	Entre dos valores (ambos incluidos)	`edad BETWEEN 18 AND 65`
`IN`	Coincide con un valor del conjunto	`ciudad IN ('Madrid', 'Barcelona')`
`NOT IN`	No coincide con los valores	`estado NOT IN ('cancelado', 'pendiente')`
`LIKE`	Coincide con patrón de caracteres	`nombre LIKE 'A%'`
`NOT LIKE`	No coincide con patrón	`email NOT LIKE '%@gmail.com'`
`IS NULL`	El valor es nulo	`telefono IS NULL`
`IS NOT NULL`	El valor no es nulo	`email IS NOT NULL`

---

10. OPERADORES LÓGICOS

Operador	Significado	Regla
AND	Y lógico	TRUE si todas las condiciones son TRUE
OR	O lógico	TRUE si alguna de las condiciones es TRUE
NOT	Negación	Invierte el resultado (TRUE ↔ FALSE)

---

11. **ORDEN DE EVALUACIÓN DE OPERADORES**

Prioridad	Operador
1	Operadores aritméticos (/, *, -, +)
2	Operadores de concatenación
3	Operadores de comparación (=, >, >=, <, <=, <>)
4	IS NULL, LIKE, IN
5	BETWEEN
6	NOT
7	AND
8	OR

---

12. ORDENACIÓN - ORDER BY

Definición de ORDER BY

Ordena los resultados de una consulta según una o más columnas.

Sintaxis para ordenación

```
SELECT columna1, columna2, ...
FROM nombre_tabla
WHERE condición
ORDER BY columna1 ASC, columna2 DESC;
```

Opciones disponibles

Parámetro	Significado
ASC	Orden ascendente (de menor a mayor) - Por defecto
DESC	Orden descendente (de mayor a menor)

---

13. CONSULTAS RESUMEN - FUNCIONES DE AGREGACIÓN

Definición de agregación

Consultas que devuelven un único valor resumido en lugar de filas individuales. Incluyen operaciones como: totales, conteos, promedios, máximos, mínimos.

Función de agregacion según tipo

Función	Valor Devuelto	Notas
AVG()	Media aritmética	Ignora valores NULL
COUNT()	Cantidad de registros	Cuenta NULL, duplicados
MAX()	Valor máximo	Ignora NULL; válido para números, textos y fechas
MIN()	Valor mínimo	Ignora NULL; válido para números, textos y fechas
SUM()	Suma total	Ignora NULL; solo para números
STDDEV()	Desviación estándar	Ignora NULL; solo para números
VARIANCE()	Varianza	Ignora NULL; solo para números

Variantes de COUNT()

```
-- Cuenta TODAS las filas, incluyendo NULL
COUNT(*)
-- Cuenta solo filas donde columna NO es NULL
COUNT(columna)
-- Cuenta valores DIFERENTES (sin NULL)
COUNT(DISTINCT columna)
```

Ejemplo de agregación

```
SELECT COUNT(*) AS total_ciudades,
       AVG(Population) AS poblacion_media,
       MAX(Population) AS poblacion_maxima
FROM city;
```

---

14. AGRUPAMIENTO - GROUP BY

Agrupamiento de datos

Divide las filas en grupos según los valores de una o más columnas, y aplica funciones de agregación a cada grupo.

Construcción de GROUP BY

```
SELECT columna_agrupación, FUNCION_AGREGADO(columna)
FROM nombre_tabla
GROUP BY columna_agrupación;
```

Reglas importantes

- Si usas función de agregación en SELECT: Todas las columnas no agregadas deben estar en GROUP BY
- Orden de evaluación: WHERE → GROUP BY → Función agregada
- No se permiten alias: No puedes usar alias en la cláusula GROUP BY

Caso 1 - Una columna

```
SELECT id_cliente, SUM(total) AS total_ventas
FROM pedido
GROUP BY id_cliente;
```

Caso 2 - Varias columnas

```
SELECT id_cliente, fecha, SUM(total) AS total_ventas_fecha
FROM pedido
GROUP BY id_cliente, fecha;
```

Resultado: Suma de ventas para cada cliente en cada fecha diferente.

---

15. FILTRADO DE GRUPOS - HAVING

Filtrado de grupos

Filtro aplicado después de agrupar, equivalente a WHERE pero para grupos. Descarta grupos que no cumplan la condición.

Construcción de HAVING

```
SELECT columna_agrupación, FUNCION_AGREGADO(columna)
FROM nombre_tabla
GROUP BY columna_agrupación
HAVING condición_grupo;
```

Diferencia: WHERE vs HAVING

Cláusula	Aplica A	Momento
WHERE	Filas individuales	Antes de agrupar
HAVING	Grupos de filas	Después de agrupar

Ejemplo de HAVING

```
SELECT id_cliente, SUM(total) AS total_ventas
FROM pedido
GROUP BY id_cliente
HAVING SUM(total) > 1000;
```

Resultado: Solo clientes cuyas ventas totales superen 1000.

---

16. CONSULTAS SOBRE VARIAS TABLAS - JOINS

Uniones de tablas

Extrae datos de múltiples tablas relacionadas mediante condiciones de unión.

Clasificación de uniones disponibles

A. COMPOSICIONES INTERNAS (INNER JOIN)

Devuelven solo las filas con coincidencia en ambas tablas.

1. NATURAL JOIN (automático)

```
SELECT *
FROM tabla1
NATURAL JOIN tabla2;
```

- Características
  - Usa automáticamente todas las columnas con mismo nombre y tipo en ambas tablas
  - Cómodo pero automático

2. USING (columnas comunes)

```
SELECT t1.columna1, t2.columna2, columna_comun
FROM tabla1 t1
JOIN tabla2 t2 USING (columna_comun);
```

- Características:
  - Especifica qué columnas usar para unir
  - Útil cuando hay múltiples columnas comunes pero quieres usar solo algunas

3. ON (máxima flexibilidad)

```
SELECT t1.columna1, t2.columna2
FROM tabla1 t1
JOIN tabla2 t2 ON t1.id = t2.id_tabla1;
```

- Características:
  - Máxima flexibilidad
  - Especifica exactamente la condición de unión
  - Funciona independientemente de nombres de columnas

4. CROSS JOIN (producto cartesiano)

```
SELECT t1.columna1, t2.columna2
FROM tabla1 t1
CROSS JOIN tabla2 t2;
```

- Características
  - Combinación de todas las filas de una tabla con todas las filas de la otra
  - Producto cartesiano (sin condición de unión)
  - Cuidado: puede generar muchas filas

B. COMPOSICIONES EXTERNAS (OUTER JOIN)

Devuelven todas las filas de una o ambas tablas, completando con NULL donde no hay coincidencia.

1. LEFT JOIN (tabla izquierda)

```
SELECT t1.columna1, t2.columna2
FROM tabla1 t1
LEFT JOIN tabla2 t2 ON t1.id = t2.id_tabla1;
```

- Características
  - Todas las filas de tabla1 (izquierda)
  - Columnas de tabla2 completadas con NULL si no hay coincidencia
  - "Tabla izquierda" = primera en FROM

2. RIGHT JOIN (tabla derecha)

```
SELECT t1.columna1, t2.columna2
FROM tabla1 t1
RIGHT JOIN tabla2 t2 ON t1.id = t2.id_tabla1;
```

- Características
  - Todas las filas de tabla2 (derecha)
  - Columnas de tabla1 completadas con NULL si no hay coincidencia
  - Lo opuesto a LEFT JOIN

---

17. SUBCONSULTAS

Definición de subconsultas

Consulta dentro de otra consulta. Se ejecuta primero y su resultado se usa en la consulta principal.

Clasificación de subconsultas

Tipo	Devolución	Uso
Escalar	1 fila + 1 columna	Comparación simple
Fila	1+ columnas + 1 fila	Comparación de múltiples valores
Tabla	1+ columnas + 0+ filas	Comparaciones complejas

A. EN LA CLÁUSULA WHERE

```
SELECT * FROM cliente
WHERE ciudad = (SELECT ciudad FROM cliente WHERE id = 3);
```

Lógica:

1. Subconsulta obtiene la ciudad del cliente con id=3
2. Consulta principal filtra clientes de esa ciudad

B. EN LA CLÁUSULA SELECTºº

```
SELECT nombre,
       (SELECT COUNT(*) FROM pedido WHERE pedido.id_cliente = cliente.id) AS num_pedidos
FROM cliente;
```

- Características: Debe ser escalable (1 fila, 1 columna)

C. OPERADORES DISPONIBLES

1. Comparación básica

```
-- Usar con subconsultas escalares
SELECT * FROM cliente
WHERE total_pedidos > (SELECT AVG(total) FROM pedido);
```

2. IN y NOT IN

```
-- Clientes cuya ciudad está en el conjunto de ciudades
SELECT * FROM cliente
WHERE ciudad IN (SELECT ciudad FROM cliente WHERE categoria > 100);

-- Clientes que NO están en un conjunto
SELECT * FROM cliente
WHERE id NOT IN (SELECT id_cliente FROM pedido);
```

3. ALL y ANY

```
-- Mayor que TODOS los valores (= máximo)
SELECT * FROM cliente
WHERE total_pedidos > ALL (SELECT total FROM pedido);

-- Mayor que ALGUNO (= mínimo)
SELECT * FROM cliente
WHERE total_pedidos > ANY (SELECT total FROM pedido WHERE ciudad = 'Sevilla');
```

4. EXISTS y NOT EXISTS

```
-- Devuelve TRUE si la subconsulta devuelve al menos una fila
SELECT * FROM cliente
WHERE EXISTS (SELECT 1 FROM pedido WHERE pedido.id_cliente = cliente.id);

-- Clientes sin pedidos
SELECT * FROM cliente
WHERE NOT EXISTS (SELECT 1 FROM pedido WHERE pedido.id_cliente = cliente.id);
```

---

18. COMBINACIÓN DE CONSULTAS - UNION

Definición de UNION

Combina resultados de dos o más consultas en un único conjunto.

Restricciones de combinación

- Las consultas deben tener el mismo número de columnas
- Las columnas deben tener tipos de datos compatibles
- El resultado usa los nombres de columna de la primera consulta

Variante 1 - UNION (Sin duplicados)

```
SELECT columna1, columna2 FROM tabla1
UNION
SELECT columna1, columna2 FROM tabla2;
```

- Características
  - Elimina filas duplicadas automáticamente
  - Procesamiento más lento (requiere comparación)

Variante 2 - UNION ALL (Con duplicados)

```
SELECT columna1, columna2 FROM tabla1
UNION ALL
SELECT columna1, columna2 FROM tabla2;
```

- Características
  - Mantiene todas las filas, incluidos duplicados
  - Procesamiento más rápido
  - Uso cuando sabes que no hay duplicados

---

19. FLUJO DE EJECUCIÓN DE SELECT

El orden en que SQL ejecuta las cláusulas (no el orden en que escribimos):

```
1. FROM    → Selecciona las tablas base
2. WHERE   → Filtra filas individuales
3. GROUP BY → Agrupa filas
4. HAVING  → Filtra grupos
5. SELECT  → Selecciona columnas
6. ORDER BY → Ordena resultados
```

Caso de estudio completo

```
SELECT id_cliente, SUM(total) AS total_ventas
FROM pedido
WHERE estado = 'completado'
GROUP BY id_cliente
HAVING SUM(total) > 500
ORDER BY total_ventas DESC;
```

Ejecución:

1. Lee tabla `pedido`
2. Filtra solo pedidos completados
3. Agrupa por cliente
4. Calcula suma por grupo
5. Filtra grupos con total > 500
6. Muestra id_cliente y total
7. Ordena por total descendente

---

20. RESUMEN DE CONSULTAS DML

Operación	Sentencia	Función
Consulta básica	`SELECT ... FROM`	Extrae columnas
Filtrado	`WHERE`	Filtra filas
Agregación	`GROUP BY` + `HAVING`	Agrupa y filtra grupos
Ordenación	`ORDER BY`	Ordena resultados
Uniones	`JOIN`	Combina tablas
Combinación de consultas	`UNION`	Mezcla resultados
Consultas anidadas	Subconsultas	Consultas dentro de consultas

---

21. QUIZ RÁPIDO - REPASO

- DML manipula datos: SELECT, INSERT, UPDATE, DELETE
- SELECT recibe datos basándose en: Proyección (columnas), Selección (filas), Unión (tablas)
- `*` es comodín que significa: Todas las columnas
- ALIAS renombra: Columnas y tablas para mejorar legibilidad
- DISTINCT elimina: Filas duplicadas
- WHERE filtra: Filas antes de agrupar
- ORDER BY ordena: Ascendente (ASC) o descendente (DESC)
- GROUP BY agrupa + requiere: Función de agregación
- HAVING filtra: Grupos después de agrupar
- INNER JOIN devuelve: Solo filas con coincidencia
- LEFT JOIN devuelve: Todas las de la izquierda + coincidentes de la derecha
- Subconsulta escalar devuelve: 1 fila + 1 columna
- UNION elimina duplicados | UNION ALL mantiene duplicados
- EXISTS verifica: Si existen filas (TRUE/FALSE)